LOST & FOUND

Benchmark of multilingual models for matching

Binôme 3 – Matching & Resolution | Languages: English, French, Arabic | Results of 31 October 2026

1. Purpose

The matching engine must decide whether a lost declaration and a found declaration can describe the same object. Users write titles and descriptions in English, French or Arabic, sometimes mixing languages. This document compares the candidate text and photo models, describes how they were measured on our own data, reports the results, and states the choice.

Two kinds of model are needed:

- **A text embedding model** for the title and description (the only part of the score that needs language understanding).
- **An image embedding model** for the optional photo (language independent).

The other signals (date, location, category, brand, color) are rules or formulas, not models. Section 6 shows how to make them multilingual too.

> **Important:** Sizes and language coverage in sections 3 and 4 are approximate values from the model cards. Sections 7 and 8 contain results measured on our own machine on 31 October 2026: 24 text pairs, and one photo object (3 photos). They are early results: the photo test in particular covers a single object and must be repeated on about ten objects before the choice is final.

4. Photo models

Model (transformers.js id)	Type	Dims	Use in the project
Xenova/clip-vit-base-patch32	CLIP	512	Photo-to-photo similarity; light and fast. Text-to-photo works in English only.
Xenova/clip-vit-base-patch16	CLIP	512	Same family, finer image patches: more detail, about twice as slow.
Xenova/siglip-base-patch16-224	SigLIP	768	Image-text model, often better than CLIP on benchmarks.
Xenova/dinov2-small	DINOv2	384	Image only (no text). Designed to recognise the same object visually.
Xenova/dinov2-base	DINOv2	768	Larger DINOv2 variant.

Photo matching compares two pixel-based embeddings, so the language of the declaration does not matter. When one declaration has no picture (currently all of them), the photo signal is skipped and the other weights are renormalized.

5. Evaluation protocol

5.1 Dataset

Create bench/pairs.json with 20 to 30 lost/found pairs that we know describe the same object. Each pair is one lost text and one found text (title + description). Cover these situations:

#	Type of pair	Lost text	Found text
1	Same language (EN)	Black Nike backpack, main library	Black backpack with Nike logo found in the library
2	Same language (FR)	Sac à dos noir Nike, bibliothèque	Sac noir trouvé à la bibliothèque, marque Nike
3	Same language (AR)	حقيبة ظهر سوداء من نايكي في المكتبة	وجدت حقيبة سوداء بالقرب من المكتبة
4	Cross-language (AR / EN)	حقيبة ظهر سوداء في المكتبة	Black backpack found near the library
5	Cross-language (FR / EN)	Écouteurs Apple blancs avec boîtier	AirPods Pro, left earbud and case
6	Vague description	Lost my charger	White USB-C cable found in classroom A4
7	Hard negative	Black backpack, cafeteria	Black backpack, amphitheater C (different object)

Include at least five hard negatives (two similar objects that are not the same, as in case 1 of the specification) and five pairs per language combination. Add some found items with no matching lost item, so that the test also checks that the model does not match everything.

5.2 Metrics

- **Top-1 accuracy:** share of lost items whose true found item is ranked first.
- **Top-3 accuracy:** share where the true item is in the first three (what the user actually sees).
- **MRR (mean reciprocal rank):** average of 1/rank of the true item; rewards ranking it higher.
- **Separation:** average score of true pairs minus average score of hard negatives; it shows whether a threshold can work.
- **Cost:** model load time, milliseconds per text, and RAM (process.memoryUsage().rss).

5.3 Procedure

- Embed every lost text with the query prefix and every found text with the passage prefix (e5 models only).
- Compute the cosine similarity between each lost text and all found texts, then rank.
- Report the metrics per model, and also per type of pair (same language vs cross-language), because a model can be good in French and weak in Arabic.
- Run each model three times and keep the median time to reduce noise.

6. Parts of the score that are not models

The text embedding handles three languages, but the rest of the score must also understand them. Otherwise an Arabic "أسود" and an English "black" will never match.

Signal	Method	Multilingual handling
Color	Normalized comparison	Alias map: black = noir, أسود = white = blanc, أبيض = grey = gray = gris = رمادي. Best fix: use a dropdown so color is stored in one language.
Brand	Normalized comparison	Lowercase, remove accents and diacritics. Brands are mostly written the same way in all three languages.
Location	Keyword zone map	Each place has keywords in the three languages: library, bibliothèque, مكتبة; cafeteria, cafétéria, كافيتيريا.

Signal	Method	Multilingual handling
Date	exp(-gapDays / 14)	None needed. Found date before lost date gives 0.
Category	Hard filter + related map	Categories are stored as slugs (wallet_bag, clothing...), so no translation needed.

Before comparing Arabic text, normalize it: remove diacritics (tashkeel) and tatweel, and unify the letters آ / إ / أ to ا, and ى to ي.

7. Results: text models

Measured on 24 lost/found pairs (6 English, 6 French, 6 Arabic, 6 cross-language), each with a hard negative (a similar but different object). Every lost text is ranked against all 48 found texts (true matches and hard negatives, all categories mixed).

Model	Top-1	Top-3	MRR	Separation	Load (ms)	ms / text	RAM (MB)
multilingual-e5-small	58.3%	91.7%	0.748	0.041	41,394	8.2	729
paraphrase-multilingual-MiniLM-L12-v2	50.0%	91.7%	0.711	0.219	37,203	8.8	1,072
multilingual-e5-base	54.2%	91.7%	0.734	0.034	81,449	16.7	1,831
LaBSE	62.5%	87.5%	0.764	0.073	140,801	18.4	581
bge-m3 (optional)	n/a	n/a	n/a	n/a	n/a	n/a	n/a
all-MiniLM-L6-v2 (English baseline)	n/a	n/a	n/a	n/a	n/a	n/a	n/a

n/a: not measured. bge-m3 was not tested (too heavy for the target server); the English baseline all-MiniLM-L6-v2 was stopped by the system before it finished (process terminated), and it is not needed for the decision.

Top-1 by type of pair:

Model	EN-EN	FR-FR	AR-AR	Cross-language	Hard negatives
multilingual-e5-small	83%	83%	67%	0%	100%
paraphrase-multilingual-MiniLM-L12-v2	50%	50%	67%	33%	96%
multilingual-e5-base	33%	83%	83%	17%	96%
LaBSE	67%	67%	83%	33%	100%

7.1 How to read these numbers

- **Top-3 is about the same for all models (88 to 92%).** With 24 pairs, one pair is worth 4.2 points, so differences in Top-1 and Top-3 between models are within noise.
- **Separation is the clearest difference.** MiniLM-multilingual scores 0.219, against 0.03 to 0.07 for the others. Its scores for a true match and for a look-alike are far apart, so thresholds such as "high" and "medium" can be set reliably. The e5 models give all pairs similar scores, which makes fixed thresholds fragile.
- **Cross-language is the weak point of e5.** e5-small never ranks a match first when the two texts are in different languages (0%), e5-base 17%; MiniLM-multilingual and LaBSE reach 33%. This matters because the project must handle English, French and Arabic together.

- **The test is harder than the real system.** Here each text competes against all categories; in the application the candidates are first filtered by category and date, so real scores should be better for every model.
- **RAM and load time are indicative only.** The values are inconsistent with model sizes (LaBSE, the largest, shows the lowest RAM), because process memory depends on what Node has not yet released. Load times include the first download. Only ms per text is reliable.

8. Results: photo models

Five image models were run on the same machine. Speed and memory are measured on 12 photos; the quality comparison below uses one object (a grey quilted jacket) with three photos: A (worn, side view), B (the same jacket, flat, back view) and a look-alike (another grey quilted jacket, worn, back view).

8.1 Speed and memory

Model	Dims	ms / image	RAM (MB)	Load (ms)
clip-vit-base-patch32	512	92.9	278	26,013
clip-vit-base-patch16	512	212.3	290	18,447
siglip-base-patch16-224	768	198.5	310	17,304
dinov2-small	384	92.9	211	6,152
dinov2-base	768	235.3	275	90,248

All five models load and run in Node.js with @xenova/transformers. CLIP-B/32 and DINOv2-small are the fastest (about 93 ms per image); the other three take two to two and a half times longer. Load time includes the first download.

8.2 Same object versus look-alike (one object)

Similarity is the cosine between the embeddings (0 to 1). A positive gap means the model finds the real second photo of the jacket more similar than the look-alike.

Model	Same jacket (A vs B)	A vs look-alike	Gap	Verdict
clip-vit-base-patch16	0.732	0.638	+0.094	Distinguishes best
clip-vit-base-patch32	0.799	0.744	+0.054	Distinguishes
dinov2-small	0.328	0.279	+0.050	Distinguishes, low scores
siglip-base-patch16-224	0.775	0.777	-0.002	No difference
dinov2-base	0.632	0.731	-0.099	Prefers the look-alike

> **Limit:** This is a single object, so it is an example, not a measurement. The test is also biased: photo A and the look-alike are both worn by a person, while B is flat, so models that react to the overall look of the picture (DINOv2-base, SigLIP) may have been misled by the presentation rather than by the jacket. Four photo objects were prepared (jacket, black pouch, burgundy wallet, earbuds) but only the jacket has its three photos; the others still need a second photo of the same object.

9. Conclusions and decision

9.1 Text model

- **Provisional choice: paraphrase-multilingual-MiniLM-L12-v2.** It has by far the best separation (0.219), a cross-language Top-1 of 33% (e5-small: 0%), the same Top-3 as the best models and a small size (118M parameters, 8.8 ms per text).
- **Second choice: LaBSE.** Best Top-1 (62.5%) and 33% cross-language, but a lower Top-3 (87.5%), a much larger model and a slow first load.
- **Not retained: multilingual-e5-small and -base.** Narrow score range (separation 0.03 to 0.04) and weak cross-language results. e5-small would only be preferable if the application handled a single language.
- Thresholds ("high", "medium") must be set from the measured scores of the chosen model, not from fixed values such as 0.75 and 0.55.

9.2 Photo model

- **Provisional choice: CLIP-B/16 (clip-vit-base-patch16),** which has the largest gap (+0.094) on the test object. CLIP-B/32 is a close second and about twice as fast (93 ms against 212 ms); it is the fallback if speed or memory matters.
- **Not retained: SigLIP and DINOv2-base** (no gap, and a negative gap respectively on the test object). DINOv2-small distinguishes but its scores are low, which makes thresholds harder to set.
- Photo similarity is one signal among several in the final score (text, date, location, brand, color). It is skipped when one of the two declarations has no picture, which is currently the case for all 26 declarations.

9.3 What is still needed to make the choice final

- Rerun the text benchmark with the ranking restricted to the same category (as in production) and report Top-3 by language, which is the criterion of the decision rule.
- Complete the photo test: about 10 objects, each with two photos of the same object and one look-alike photographed in the same style.
- Add real declarations from the database (26 today) and real Arabic examples written by users to the text test.
- The model only proposes a match. Users accept or refuse it, and the score breakdown is stored with the match so that it can be explained to an administrator.

10. Benchmark scripts

All scripts run in Node.js with @xenova/transformers and write one JSON result file per model.

File	Role
bench/pairs.json	The 24 text pairs (group, lost, found, hardNegative).
bench/run.js <model>	Text benchmark for one model: Top-1, Top-3, MRR, separation, hard negatives, load time, ms per text, RAM, Top-1 per language group.
bench/table.js	Prints the two result tables of section 7.
bench_photo/photos/	Photos named NN_a, NN_b (same object) and NN_sosie (look-alike).
bench_photo/run_photo.js <model>	Photo benchmark for one model (needs at least 3 complete objects).
bench_photo/compare.js <model> A B [other]	Similarity of two photos, and of A with a different object, with the gap between the two.

File	Role
bench_photo/table_photo.js	Prints the photo result table.

Commands (Windows cmd, one model per process so that memory is not polluted by the others):

```
cd bench
node run.js e5-small
node run.js minilm-ml
node run.js e5-base
node run.js labse
node table.js

cd ..\bench_photo
node compare.js clip-b16 photos\01_a.png photos\01_b.png photos\01_sosie.png
node run_photo.js clip-b16
node table_photo.js
```

> **Tip:** Model keys: text = e5-small, minilm-ml, e5-base, labse, bge-m3, minilm-en; photo = clip-b32, clip-b16, siglip, dino-s, dino-b. Delete results*.json before a new series so that table.js does not show old values.